Applied Human Computer Interaction (AHCI)

Asif Fazwani

Chapter 6 – HCI in the Software Process

HCI in the Software Process

- Designing usable systems requires more than identifying good interface examples.
- We must understand the entire design process of interactive systems.
- HCI focuses on making systems usable, efficient, and user-friendly.
- The design process should follow structured development methods.

Think of apps like **Facebook or Instagram**.
Their success isn't just about features, it's about **how easy and enjoyable they are to use**.

Software Engineering and HCI

- Software Engineering manages the development of software systems. It focuses on:
- Technical processes
- Project management
- System development methods

A key concept in software engineering is the Software Life Cycle.

HCI Across the Development Process

HCI is **not just one extra step** in development.

It **influences every stage** of the software life cycle.

- Developers must constantly consider:
  - User needs
  - Ease of use
  - Interaction design

Example

When designing a **learning app**:

- Use short interactions
- Gamification
- Dark mode
- Mobile-first design

What is the Software Life Cycle?

Structured process to **design, build, and maintain software**

Ensures:

- Systematic development
- Better quality
- Reduced errors

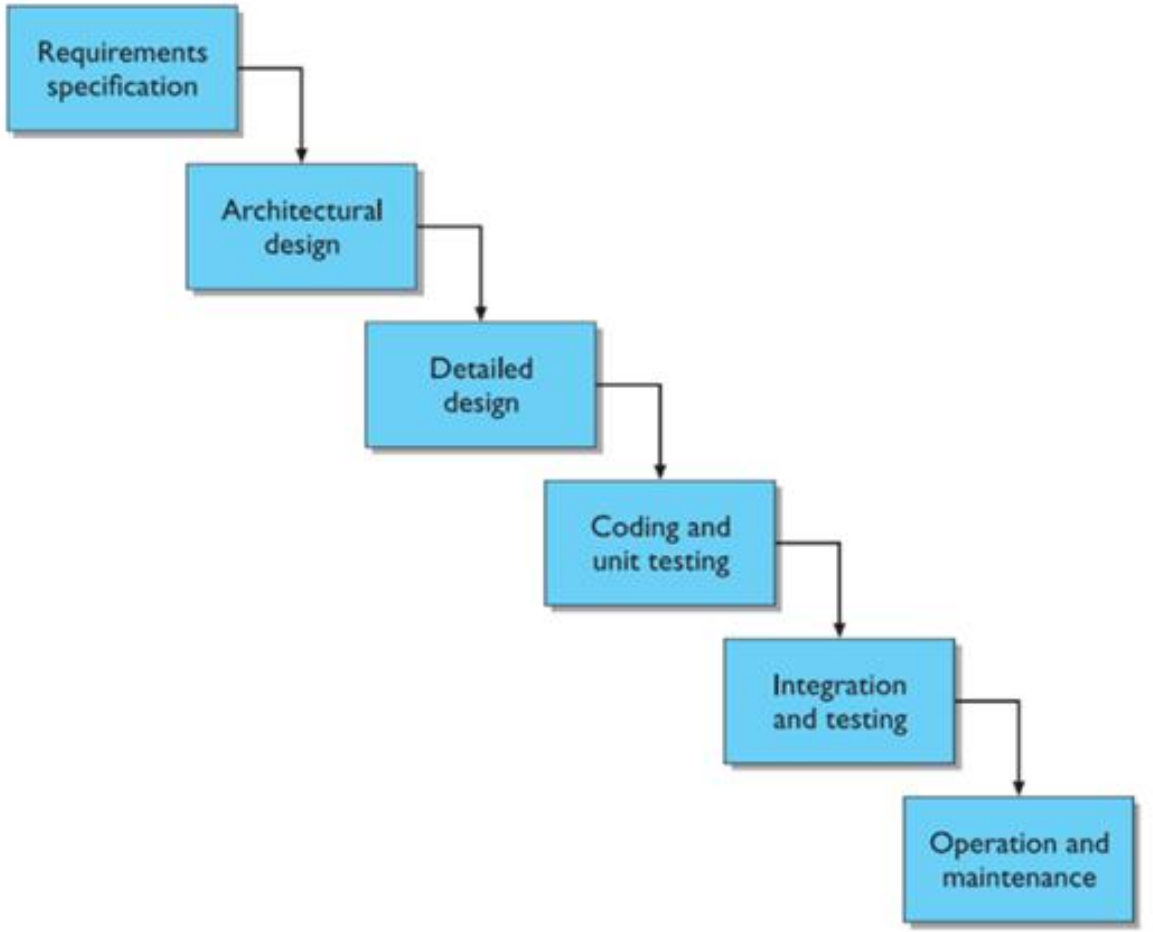
Example:
Building an app like a **study planner** → you don’t just code randomly, you follow steps (idea → design → build → test → improve)

Waterfall Model Overview

- Development flows step-by-step like a **waterfall**
- Each phase leads to the next
- But **feedback loops exist** (especially from maintenance)

Example: Like posting a video/vlog on social media:

- Idea
- Plan content
- Record
- Edit
- Upload
- Improve based on comments



Requirements Specification (WHAT?)

Requirements specification: Define **what the system should do**

- Designer and customer try capture what the system is expected to provide can be expressed in natural language or more precise languages, such as a task analysis would provide
- Gather info about: Users, Tasks, Environment
- Translate **human language → technical requirements**

Example: Designing a food delivery app:

- Users want:
  - Fast delivery
  - Live tracking
  - Easy payment

Architectural Design (HOW – High Level)

- High-level description of how the system will provide the services required factor system into major components of the system and how they are interrelated needs to satisfy both functional and nonfunctional requirements
- Break system into **major components**
- Define: System structure and Component interaction
- Focus on **big picture design**

Example: Instagram-like app:

Components:

- User profiles
- Feed system
- Messaging
- Notifications

Non-Functional Requirements

Not about features, but **how well system works**

Includes:

- Speed
- Security
- Reliability
- Usability

Example: Instagram must:

- Load videos instantly
- Not crash
- Be addictive & smooth

Detailed Design

Refinement of architectural components and interrelations to identify modules to be implemented separately the refinement is governed by the nonfunctional requirements.

- Break components into **smaller, implementable parts**
- **Decide:** Algorithms, Data structures, Logic

Example: For a chat feature:

- Message sending logic
- Notification trigger
- Read receipts

Coding & Unit Testing

- Convert design → **actual code**
- Test each part individually (**unit testing**)

Example:

- Coding login system
- Testing:
  - Correct password
  - Wrong password

Integration & Testing

- Combine all components
- Test system as a **whole**
- Includes **user acceptance testing**

Example: App works individually, but: Login + feed + messaging together → must not break

Operation & Maintenance

System is released.

Ongoing: Bug fixes, Updates, New features

Example: Apps like snapchat constantly:

Fix bugs, Add filters, Update UI

Validation vs Verification

Two key quality checks in software development:

- **Validation** → Are we building the *right product*?
- **Verification** → Are we building the *product right*?

Easy way to remember:

- **Validation = User satisfaction**
- **Verification = Technical correctness**

Example: Social Media App (like Instagram)

- **Validation (Right Product?)**
  - Users want: Easy posting, Smooth scrolling, Engaging reels
    If users find it fun and useful → Validation success
- **Verification (Product Right?)**
  - App should: Upload photos without errors, Show correct like counts, Not crash
    If everything works correctly → Verification success

Verification (Building it Right)

Ensures:

- Internal consistency
- Correct logic
- No technical errors

Happens:

- Within a stage
- Between nearby stages

Checks:

- Algorithms work correctly
- Data flows properly
- Code matches design

Validation (Building the Right Thing)

- Ensures system meets:
  - Customer needs
  - User expectations
- More **subjective** than verification
- In interactive systems: Validation = **User evaluation**

Can involve:

- Usability testing
- Real-world usage

The Formality Gap: validation will always rely to some extent on subjective means of proof

User says: “App feels slow”
But technically: System is fast → mismatch in perception

Technical vs Managerial Perspective

- Software development is not just technical, it includes **management + business decisions**
- Technical flow: **Requirements → Architecture → Detailed Design → Implementation**
- Reality: steps often **overlap or happen out of order**
- Management considers:
  - Market demand
  - Training needs
  - Team skills / outsourcing
  - Budget & deadlines

Example: Building a new social app:

- Tech team: features, UI, backend
- Management: “Will end user use it?” “Can we compete with existing app?” “Do we have AI engineers?”

Phases vs Activities

- **Activity (technical view):**
  - Design, coding, testing
- **Phase (management view):**
  - Defined by:
    - Inputs (documents)
    - Outputs (deliverables)

Sign-Off & Contracts

- Each phase ends with **approval (sign-off)**
- Signed documents create **legal/contractual obligations**

Example: Requirements document signed

- Client: “I won’t ask for extra features”
- Developer: “I will deliver everything listed”

Usability Engineering

- **Focus:** Define and measure usability explicitly
- **Introduced by:** Industry researchers (IBM, DEC)
- **Key idea:** Usability should be **engineered, not guessed**
- **Core Principle:** Decide how usability will be judged BEFORE building the system

Example: When designing a student app:

Not just “easy to use”

But: “Users can submit assignment in ≤ 3 clicks”

Why Usability Needs Measurement

The ultimate test of usability based on measurement of user experience.

Usability engineering demands that specific usability measures be made explicit as requirements.

- **Usability is based on:** Real user experience
- **Focus often on:** Interface (buttons, screens)
- But true usability includes:
  - System architecture
  - User thinking (cognitive load)

Key Insight: UI ≠ Full experience

Limitation of Usability Engineering

- **Easy to measure:** Clicks, time, actions
- **Hard to measure:**
  - Thinking effort
  - Confusion
  - Mental load

Key Problem: Not everything important is **measurable**

Example Valorant:

- Easy to measure: reaction time
- Hard to measure: decision stress in gameplay

Usability Specification

- A **user-centered design approach**
- Focuses on **defining measurable usability goals**
- Ensures systems are judged based on **real user experience**

Example: Apps like TikTok optimize usability by tracking:

- Watch time
- Click behavior
- Ease of interaction (scroll, like, share)

Usability Specification

Part of **requirements phase**

Defines: How usability will be evaluated

Each usability attribute includes 6 elements:

- Measuring concept
- Measuring method
- Now level
- Worst case
- Planned level
- Best case

Usability Specification

Example Attribute – Recoverability

- **Recoverability = ability to fix mistakes**
- Includes:
  - Undo
  - Redo
  - Error correction

Example: Undo sending message in WhatsApp

- Edit caption after posting on Instagram

If users can’t recover → frustration + abandonment

Case Study – VCR Problem

- **Old VCR systems were:**
  - Hard to program
  - Easy to mess up
- No undo → users had to:
  - Restart everything
  - Remember previous settings

Modern equivalent:

- Booking a flight and losing all entered data after one error

Usability Specification - Case Study Cont.

Attribute: Backward recoverability

Measuring concept: Undo an erroneous programming sequence

Measuring method: Number of explicit user actions to undo current program

Now level: No current product allows such an undo

Worst case: As many actions as it takes to program-in mistake

Planned level: A maximum of two explicit user actions

Best case: One explicit cancel action

Usability Specification

1. **Measuring Concept** *What are we trying to enable?*
   Undo a wrong programming sequence
   Example: You set something wrong → you can reverse it
2. **Measuring Method** *How do we measure success?*
   Count how many steps (clicks/taps) it takes to undo
   Example: 1 tap = great, 6 taps = frustrating
3. **Now Level** *What's the current situation?*
   No product currently allows undo
   Meaning: Users are stuck if they make a mistake

Usability Specification

4. **Worst Case** *Minimum acceptable usability*
   Undo takes as many steps as doing the whole task again
   Example: You mess up → must redo everything from scratch.
5. **Planned Level** *Target goal for designers*
   Undo should take **max 2 actions**
   Example: Tap “Back” → confirm → done
6. **Best Case** *Ideal scenario*
   Undo in **1 action**
   Example: One tap “Undo”

ISO usability standard 9241

An international standard that defines **what “good usability” means”**

It says usability has **3 key parts**:

- Effectiveness
- Efficiency
- Satisfaction

Effectiveness: Did it work?
Efficiency: Was it easy/fast?
Satisfaction: Did you enjoy it?

ISO usability standard 9241

1. Effectiveness *Can users successfully achieve their goal?*

- Focus: **Accuracy + completion**
- Did the user actually finish the task?

Example: Using Yango

- Can you successfully book a ride?
- If app crashes before booking → Not effective
- Good UX = Task completed successfully

ISO usability standard 9241

2. Efficiency: *How much effort/time does it take?*

- Focus: **Speed + effort**
- Even if task is possible, is it easy?

Example: Ordering food on DoorDash

- 3 taps to reorder favorite meal → Efficient
- 10 screens + re-enter details → Inefficient
- Good UX = Fast and low effort

ISO usability standard 9241

3. Satisfaction: *How does the user feel while using it?*

- Focus: **Enjoyment, comfort, frustration**
- Emotional experience matters

Example: Scrolling TikTok

- Smooth, fun, addictive → High satisfaction
- Laggy, confusing UI → Low satisfaction
- Good UX = Feels good to use

Usability objective	Effectiveness measures	Efficiency measures	Satisfaction measures
Suitability for the task	Percentage of goals achieved	Time to complete a task	Rating scale for satisfaction
Appropriate for trained users	Number of power features used	Relative efficiency compared with an expert user	Rating scale for satisfaction with power features
Learnability	Percentage of functions learned	Time to learn criterion	Rating scale for ease of learning
Error tolerance	Percentage of errors corrected successfully	Time spent on correcting errors	Rating scale for error handling

Activity 6

https://tinyurl.com/Spring2026-Activity6

Q1: Based on the metrics, is the design successful?

Response:

Yes, because:

- High task completion
- Fast interaction
- Meets defined usability goals

It meets usability engineering metrics

Q2: Despite meeting metrics, what problems do users face?

Responses:

- Users make accidental posts
- Lack of confirmation/review step
- System is too fast → causes errors
- Stressful experience

Good metrics ≠ good experience

Q3. What assumption did the designers make?

Responses:

- Faster is always better
- Fewer steps = better usability
- Users prefer speed over control
- Errors can be fixed using undo

Q4. Are the designers solving the right problem?

Responses: No

- They focused on **undo (fixing errors)**
- Instead of **preventing errors**

Design focuses on recovery, not prevention

Q5: How would you improve the design?

Responses:

- Add preview before posting
- Add confirmation (“Are you sure?”)
- Allow editing before posting
- Slow down the process slightly
- Add “hold to post” instead of single tap

Balance speed + control

Prevent mistakes, not just fix them

ITERATIVE DESIGN AND PROTOTYPING

What is Iterative Design?

- Requirements **can't be fully known at the start**
- Build → Test → Improve → Repeat
- Focus: **continuous improvement based on real user feedback**

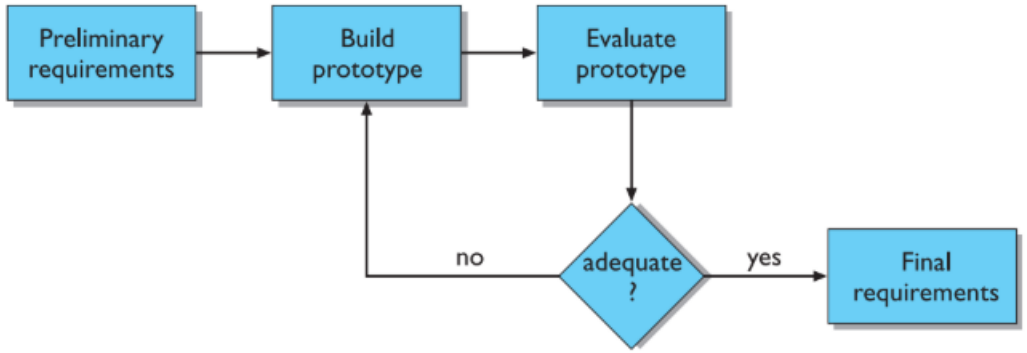
Simply: You don't get it perfect the first time, you refine it step by step.

Why Iterative Design?

- Users behave differently than expected
- Early assumptions are often wrong
- Testing reveals **real problems**

Key Idea: "Design is guessing → Testing is truth"

ITERATIVE DESIGN AND PROTOTYPING



What is Prototyping?

- Prototype = **early version of a system**
- Simulates **some features, not all**
- Used for **testing ideas quickly**

Simply: Like a rough draft before the final version.

Throw-away Prototyping

- Build → Test → Learn → **Discard**
- Final system built **from scratch using insights**

When used: When exploring ideas or unclear requirements

Example: Startup creates a fake landing page to test demand, then builds real app later.

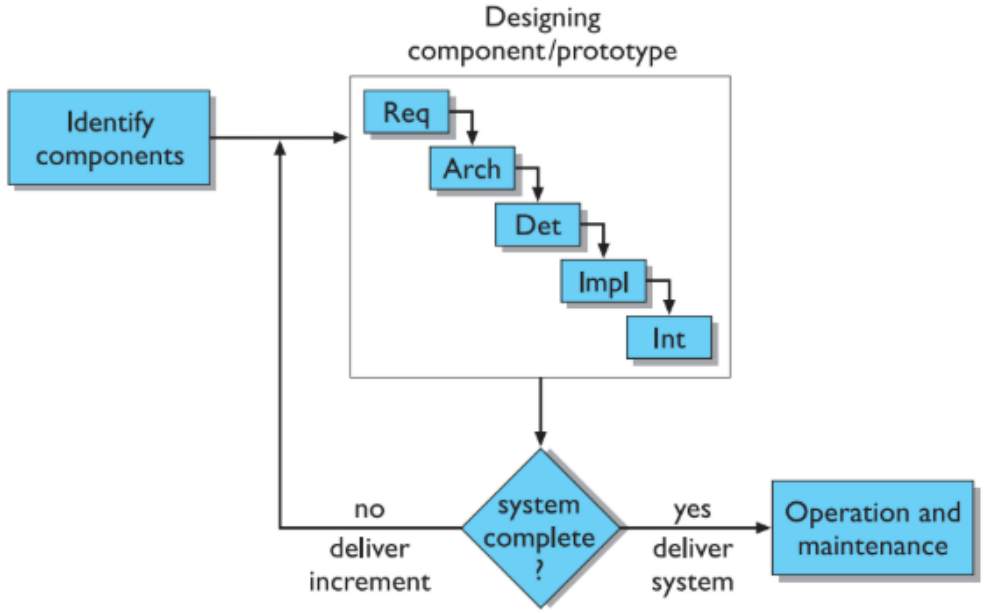
Incremental Prototyping

- System built **in parts (modules)**
- Each release adds new features
- Final system = combination of all parts

Simply: Build it piece by piece.

Example: WhatsApp:

- First: messaging
- Then: voice calls
- Then: video calls
- Then: status/stories



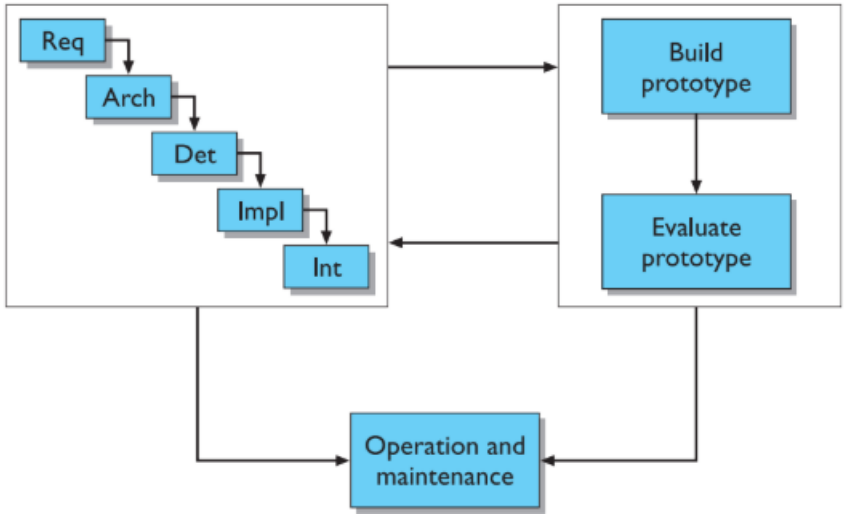
Evolutionary Prototyping

- Prototype is **continuously improved**
- Not discarded, it **becomes the final system**

Simply: Start small → grow into full product

Example: AI tools like ChatGPT:

- Started simple
- Gradually improved with new features (voice, memory, multimodal)



Management Challenge

Time: Prototyping takes time, Throw-away prototypes may feel “wasted effort” , Need **rapid prototyping**

Risk: Fast building ≠ Proper testing

Planning: Hard to estimate Time, Cost, Effort

Many managers lack experience in iterative design

Non-Functional Issues: Prototypes often ignore

- Performance
- Security
- Reliability

Problem: These are critical in real systems

Example: A smooth-looking app prototype may lag badly in real use → users uninstall instantly

Quick question??

Which apps improved over time vs failed due to bad design?

Have you ever deleted an app after one bad experience? Why?

Prototyping Techniques

- Different ways to create **rapid prototypes**
- Range from **simple sketches → interactive simulations**
- Goal: **test ideas quickly with users**
- Not all prototypes are coded; some are just drawings.

Storyboards (Basic Prototyping)

- Visual snapshots of the interface
- No real functionality
- Shows **user flow step-by-step**

Example: Designing a food delivery app:

- Screen 1: Browse food
- Screen 2: Add to cart
- Screen 3: Checkout

Prototyping Techniques

Storyboards – Modern Version

- Can include **basic animations**
- Shows how interaction flows over time

Benefit: Helps visualize user experience before coding

Example: Snapchat story UI mockup showing swipe gestures

Storyboards Limitation

- No real interaction
- Can’t test actual usability
- Only shows appearance, not behavior

Example: A UI may look cool in design, but confusing when actually used

Prototyping Techniques

Limited Functionality Simulation

- Some real interaction is added
- Mimics actual system behavior
- Users can **click, type, interact**

Simply: Fake app, but partially working

Example: A prototype where buttons work, but backend isn't connected

Simulation Example

- Users interact using mouse/keyboard
- System responds with **predefined actions**

Concept: Simulate real use without building full system

Example: Clicking "Like" button changes icon, but doesn't actually store data

Design Rationale

- Explains **why a system is designed the way it is**
- Includes:
  - Structure (how it's built)
  - Behavior (how it works)
- Not just what you built—but why you built it that way

Example: Why Instagram moved from chronological feed → algorithm feed (engagement > timeline)

Benefits of design rationale

- Communication throughout life cycle
- Reuse of design knowledge across products
- Enforces design discipline
- Presents arguments for design trade-offs
- Organizes potentially large design space
- Capturing contextual information

Benefits of design rationale

Communication throughout life cycle

- Not a single phase in development
- Happens **throughout the lifecycle**
- Includes: Reflection (thinking), Documentation (recording decisions)

Key Idea: Design rationale = ongoing thinking + recording

- Helps teams understand past decisions
- Prevents wrong assumptions later
- Supports collaboration

Example: New developer joins a startup → understands why dark mode was prioritized

Benefits of design rationale

Knowledge Reuse

- Past design decisions can be reused
- Helps in similar future projects
- “Don’t reinvent the wheel”

Example: Using TikTok-style vertical scrolling in new apps because it already works

Enforces design discipline - Better Decision-Making

- Forces designers to think carefully
- Encourages: Comparing options and Justifying choices

Example: Choosing between:

- Bottom navigation bar
- Specific category menu

Benefits of design rationale

Design trade-offs - No Single “Best” Design

- Multiple valid design options
- Trade-offs are required

Example Trade-off:

- Buttons visible → easy to use
- Menu hidden → saves space

Example: Spotify: Simple UI vs advanced controls (some features hidden)

Types of Design Rationale

1. **Process-Oriented**
   Records decision history
   Used during design

- **Idea:** Tracks *what decisions were made* + *why* + *in what order*

Example: Designing a Food Delivery App

- Team first chose **burger menu**
- Users found it confusing
- Switched to **bottom navigation bar**
- Reason recorded: “Users prefer visible options for faster access”

“Tracks the *story* of design decisions step by step”

Types of Design Rationale

2. Structure-Oriented

Focus on all possible design options
Not focused on history

Idea: Shows all possible design options (not history)

Example: Login System Design Possible options:

- Email + Password
- Google Sign-In
- Phone OTP
- Face ID

Analysis: OTP = easy but less secure , Google = fast signup , Password = traditional

“Compares all options before choosing one”

Types of Design Rationale

3. Psychological (User-Focused)

Focus on user behavior & experience

Idea: Based on how users think, behave, and feel

Example: Infinite Scroll (like TikTok/Instagram Reels)

- Users prefer **continuous content**
- Reduces effort (no clicking “next”)
- Increases engagement/addiction

Reasoning: “Users stay longer when content is effortless to consume”

Issue-based information system (IBIS)

IBIS = Issue-Based Information System

- Developed in the **1970s**
- Used to structure **design discussions**

Idea: Break design into: Questions, Possible answers, Supporting/opposing reasons

Issue → The problem/question

Position → Possible solution

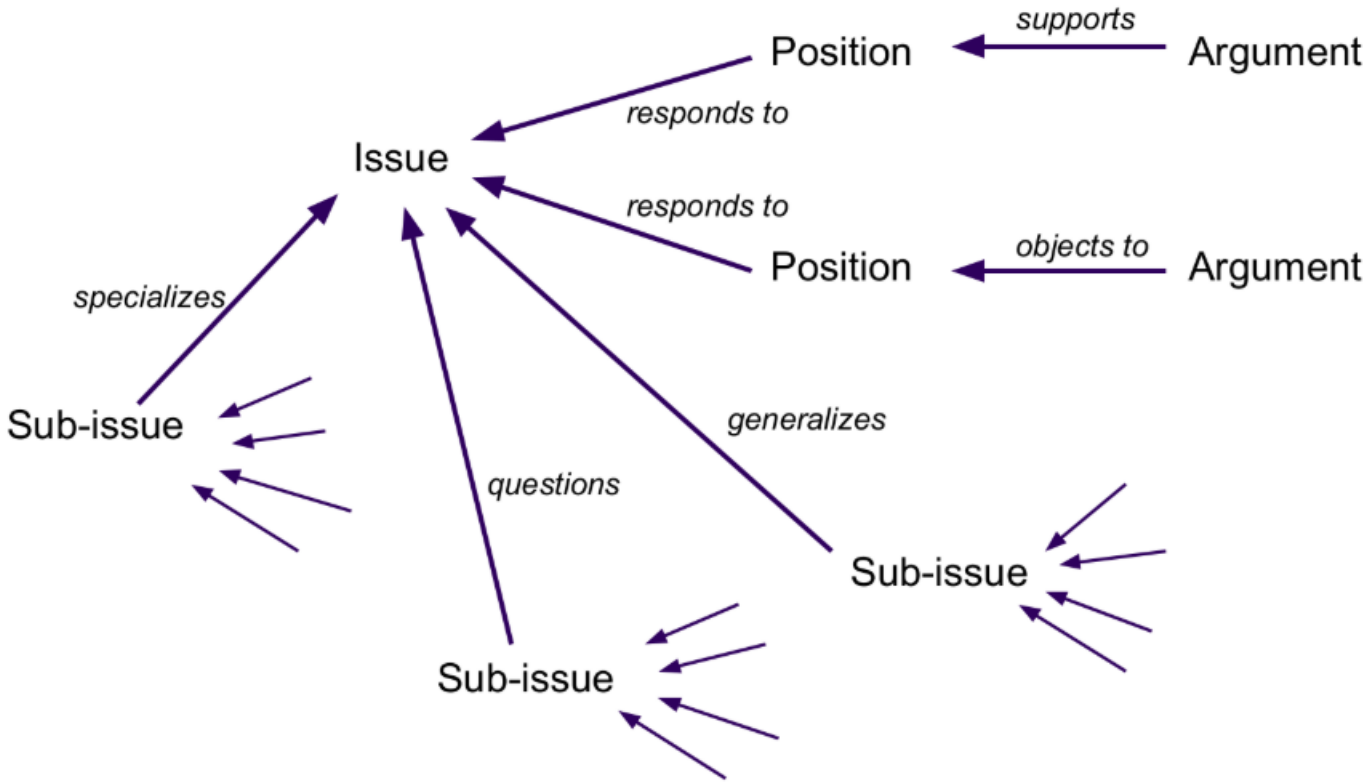
Argument → Supports or opposes solution

Example (IBIS in Action)

- **Issue:** How should a social media app show content?
- **Positions:** Show chronological feed, Show algorithm-based feed
- **Arguments:** Algorithm → more engaging , Algorithm → less control for users

Issue-based information system (gIBIS – Graphical Version)

- gIBIS is a graphical version
- **Issue (center)** → main question
- **Positions** → possible answers branching out
- **Arguments** → support or oppose positions
- **Sub-issues** → smaller related problems



Issue-based information system (IBIS)

Example Using gIBIS

Issue: Should we add dark mode?

Positions: Yes or No

Arguments:

- Yes → reduces eye strain
- No → increases design complexity

Sub-issue: Should dark mode be default or optional?

Design Space Analysis

Focus: **Exploring all possible design options**

Done **after design (post hoc)**

Helps structure decisions clearly
Instead of tracking history → organize all possible choices

QOC Notation (Core Idea)

- **Q → Questions** (design problems)
- **O → Options** (possible solutions)
- **C → Criteria** (how we evaluate options)
- Question → Options → Evaluate using Criteria

Design Space Analysis

Left: **Question** (main problem)

Middle: **Options** (different solutions)

Right: **Criteria** (evaluation factors)

- Lines:
  - Solid → positive (good)
  - Dashed → negative (bad)
- Highlighted box → **best option**
  It's like a **decision comparison map**

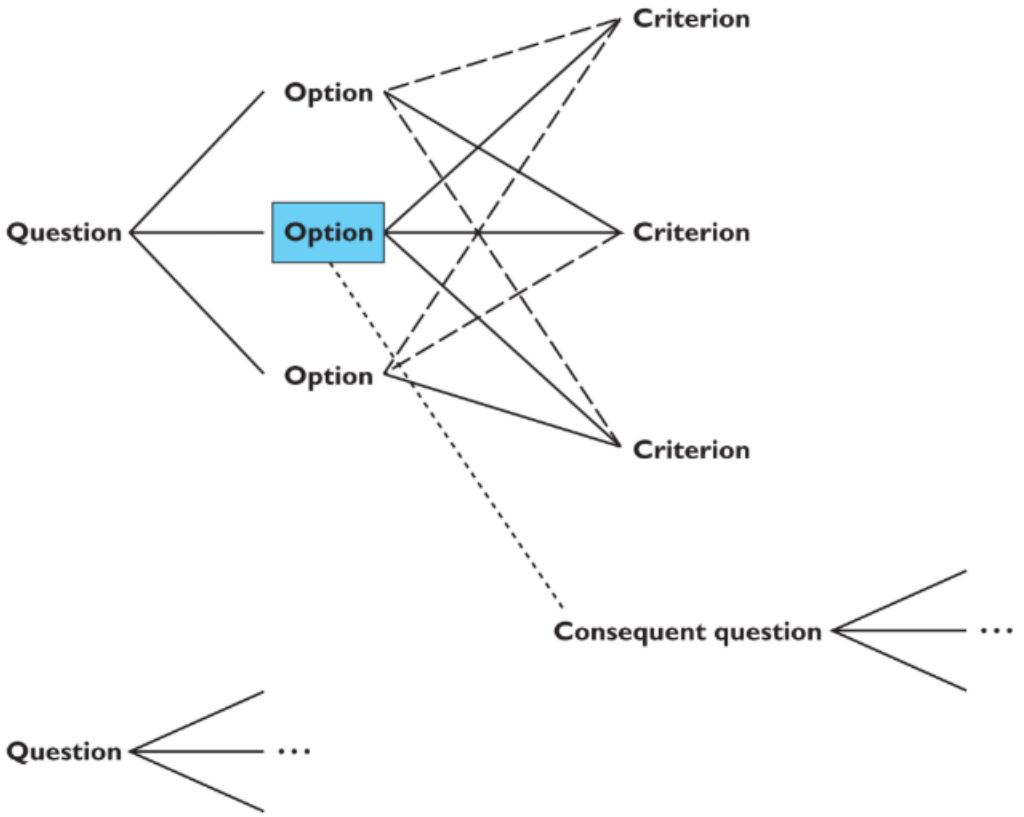


Figure 6.9 The QOC notation

Design Space Analysis - Example

Question: How should a video app show content?

Options:

- Infinite scroll
- Pagination (pages)
- Load more button

Criteria:

- Ease of use
- Engagement
- Control

Design Space Analysis - QOC

Challenges and Limitations:

- Choosing:
  - Right **questions**
  - Right **criteria**
- Bad criteria = bad decision

Example: If you ignore user addiction, you may choose wrong UI

Limitation of QOC

- Doesn’t show: Which criterion is more important
- No clear “winner” sometimes
- It compares, but doesn’t always decide

Psychological Design Rationale

- Focus: **How users think, behave, and experience the system**
- Explains **impact of design on users**
- Based on **user tasks** Not “why we designed it” → but “how it affects users”

Example: Why TikTok uses autoplay → reduces effort → increases engagement

- to support task-artefact cycle in which user tasks are affected by the systems they use
- aims to make explicit consequences of design for users
- designers identify tasks system will support
- scenarios are suggested to test task
- users are observed on system
- psychological claims of system made explicit
- negative aspects of design can be used to improve next iteration of design

Summary

The software engineering life cycle

distinct activities and the consequences for interactive system design

Usability engineering

making usability measurements explicit as requirements

Iterative design and prototyping

limited functionality simulations and animations

Design rationale

recording design knowledge

process vs. structure